

Lab 3 - Instructor's View

Learning objectives

By the end of this lab, you should be able to:

- Recognize common object types in R and explain what they are used for.
- Write and call simple functions with arguments.
- Use conditional statements and loops to control program flow.
- Read a dataset into R and summarize key variables.
- Create a basic plot to visualize differences across groups over time.

Recap

- Objects: things we work with in R
- Functions: actions we perform on objects
- Control flow: the logic and sequence of actions

1. Objects

- Integers
- Floats
- Characters
- Vectors
- Factors
- Data frames

```
1L
```

```
[1] 1
```

```
1.2
```

```
[1] 1.2
```

```
"apple"
```

```
[1] "apple"
```

```
c(1,2,3)
```

```
[1] 1 2 3
```

```
factor(c(0, 1, 0), levels = c(0, 1), labels = c("control", "treated"))
```

```
[1] control treated control  
Levels: control treated
```

```
temp_df <- data.frame(apple = c(1, 2), pear = c(2, 2))
```

Objects: The types of instances

```
class(temp_df)
```

```
[1] "data.frame"
```

```
typeof(temp_df)
```

```
[1] "list"
```

```
str(temp_df)
```

```
'data.frame':  2 obs. of  2 variables:  
 $ apple: num  1 2  
 $ pear : num  2 2
```

2. Functions

- Change the status of existing objects
- Create new objects

3. Control flow

- Sequential execution
- Conditional statements
- Loops

In R, control flow tells the program in what order to execute commands and when to repeat or skip actions.

Sequential execution

```
vec_1 <- c(1, 2, 3)
vec_2 <- c(90, 2, -1)

num_values <- length(vec_1) + length(vec_2)

cat("There are", num_values, "values in total.")
```

There are 6 values in total.

Conditional statements

```
raining = TRUE

if (raining == TRUE){
  cat("It is raining!")
} else{
  cat("It is not raining.")
}
```

It is raining!

Loops

$$f(x_1, \dots, x_n) = \sum_{i=1}^n x_i^2$$

```
vec <- 1:10

sum_squared_vec = 0
for (value in vec){
  sum_squared_vec = sum_squared_vec + value^2
}

cat("The sum of squared values in vec is:", sum_squared_vec)
```

The sum of squared values in vec is: 385

Block 0: Warm-up

Try it on your own.

This block reviews the basics of object creation and data manipulation. Make sure you can render the file successfully before moving on.

[3 mins]

```
countries <- data.frame(
  country = c("Japan", "India", "China", "Vietnam", "Afghanistan",
              "Egypt", "Kenya", "Nigeria", "Botswana", "Ethiopia"),
  continent = c("Asia", "Asia", "Asia", "Asia", "Asia",
                "Africa", "Africa", "Africa", "Africa", "Africa"),
  life_1952 = c(63.0, 37.4, 44.0, 40.4, 28.8, 41.9, NA, 36.3, 47.6, 34.1),
  life_2007 = c(82.6, 64.7, 73.0, 74.2, 43.8, 71.3, 54.1, 46.9, 50.7, 52.9)
)
```

```
# 1. Calculate the mean of life_1952. If you get NA, fix it.  
mean_life_1952 <- mean(  
  countries$life_1952,  
  na.rm = TRUE  
)  
  
cat("Mean life_1952:", mean_life_1952, "\n")
```

Mean life_1952: 41.5

```
# 2. Show only the African rows  
n_africa_rows <- nrow(countries[countries$continent == "Africa",])  
  
cat("Number of African countries:", n_africa_rows)
```

Number of African countries: 5

Block 1: Functions

This section introduces functions, which let us package repeated steps into a reusable recipe.

1. Define a function

A function is also an object.

```
# We define a vector: c(1, 2, 3)  
temp_vec <- c(1, 2, 3)  
temp_vec
```

[1] 1 2 3

```
# We define a function: sum_squared.  
func_hello_world <- function(){  
  cat("Hello world!")  
}
```

```
func_hello_world()
```

Hello world!

```
print_hello_world <- func_hello_world  
  
print_hello_world
```

```
function ()  
{  
  cat("Hello world!")  
}
```

```
mean(temp_vec)
```

[1] 2

```
compute_mean <- mean  
  
compute_mean(temp_vec)
```

[1] 2

```
compute_mean
```

```
function (x, ...)  
UseMethod("mean")  
<bytecode: 0x11c74f518>  
<environment: namespace:base>
```

```
# Functions have a name.
```

Lesson: Do not use variable names that could be confused with function names.

2. Call a function

What happens when you call a function?

- The code is executed within the scope of the function.

Why would the following code raise an error?

```
sum_squared <- function(vec){  
  return(sum(vec^2))  
}  
  
sum_squared()
```

Error in `sum\_squared()`:
! argument "vec" is missing, with no default

Predict whether the following code would throw an error.

```
result_of_1_plus_2 <- function(){  
  return(1 + 2)  
}  
  
result_of_1_plus_2()
```

[1] 3

3. Arguments in a function

- Positional arguments
- Required arguments
- Default arguments / optional arguments

From Fahrenheit to Celsius:

$$^{\circ}C = \frac{(^{\circ}F - 32) \times 5}{9}$$

```
# Define the conversion
f_to_c <- function(f_degree = 100, round_to_int = FALSE){
  if (round_to_int){
    return(as.integer(((f_degree - 32) * 5) / 9))
  }
  else{
    return(((f_degree - 32) * 5) / 9)
  }
}

# Set a default value of 100
f_to_c(round_to_int = TRUE)
```

[1] 37

```
# Set an argument called "round_to_int: bool"
f_to_c()
```

[1] 37.77778

Question 1: Why do we get an error if we leave **round_to_int** undefined?

Question 2: Why do we not get an error when we do not set the value of **f_degree**?

```
# boiling: 212
f_to_c(212)
```

[1] 100

```
# body temperature: 100
f_to_c(100)
```

[1] 37.77778

```
# works on a whole vector at once: 32, 50, 100, 212  
f_to_c()
```

```
[1] 37.77778
```

```
f_to_c(f = 100)
```

```
[1] 37.77778
```

Recommended practice:

- Pass arguments by name.
- Check the default values if you want to be especially careful.

Common mistakes to watch for:

- Forgetting to provide required arguments.
- Using the wrong order of arguments when not naming them.
- Confusing object names with function names.

Try it

[4 mins]

```
c_to_f <- function(celsius){  
  return(((celsius * 9) / 5) + 32)  
}
```

1. Fill in the skeleton and run the chunk so R learns the recipe.

2. Call it for 100 and for 37.

```
cat("c to f: 100:", c_to_f(100), "\n")
```

```
c to f: 100: 212
```

```
cat("c to f: 37:", c_to_f(37), "\n")
```

c to f: 37: 98.6

```
# 3. Call it on the vector c(0, 20, 30)
cat("c to f: 0, 20, 30:", c_to_f(c(0, 20, 30)))
```

c to f: 0, 20, 30: 32 68 86

Block 2: Reading a data file

This section shows how to organize your files and load data into R so you can start analyzing it.

Step 1. Make a folder. On your Desktop or in Documents, create a folder named exactly `gov50-section5`, all lowercase, with no spaces.

Step 2. Save this file into it. In RStudio, choose **File > Save As**, navigate to `gov50-section5`, and save the file there. The copy you downloaded is likely in your Downloads folder. From now on, work in the copy inside `gov50-section5`.

Step 3. Get the data file. Go to the Ed forum and download [nominate.csv](#). Your browser saves it in Downloads. Drag it into `gov50-section5`, next to this file. If it opens in Excel or Numbers instead, close it without saving and find the file in Downloads.

Step 4. Point R to the folder. R always has a **working directory**, which is the folder it looks in when you give it a file name. `getwd()`, short for "get working directory," shows which folder that is. Click in the Console at the bottom of RStudio, type the following, and press Enter:

```
getwd()
```

Step 5. Check, then render. Run this chunk. Both this `.Rmd` and `nominate.csv` should appear in the list.

```
list.files()      # the files in the working directory
```

```
[1] "lab-instructor_files"      "lab-instructor.html"
[3] "lab-instructor.qmd"       "lab-instructor.rmarkdown"
```

```
[5] "lab-student.html"      "lab-student.Rmd"
[7] "nominate.csv"
```

Step 6. Read the file and look at it.

```
nominate <- read.csv("./nominate.csv")
str(nominate)
```

```
'data.frame':  31663 obs. of  7 variables:
 $ year      : int  1879 1879 1879 1879 1879 1879 1879 1879 1879 1879 ...
 $ id        : int  17 51 71 74 172 229 280 282 320 345 ...
 $ name      : chr  "ACKLEN, Joseph Hayes" "AIKEN, David Wyatt" "ALDRICH, Nelson Wilmarth"
 "ALDRICH, William" ...
 $ state     : chr  "LA" "SC" "RI" "IL" ...
 $ district  : int  3 3 1 1 1 7 14 8 10 16 ...
 $ party     : chr  "Democrat" "Democrat" "Republican" "Republican" ...
 $ score     : num  -0.256 -0.308 0.401 0.457 0.299 -0.604 -0.345 -0.501 -0.265 0.351 ...
```

Column	What it holds
<code>year</code>	The first year of that Congress: 1879, 1881, ..., 2025
<code>id</code>	A number for each member, the same in every Congress they served. A member who switches party gets a new one.
<code>name</code>	The member's name, last name first
<code>state</code>	A two-letter state abbreviation
<code>district</code>	The district number within the state; 0 marks a non-voting delegate from DC or a territory
<code>party</code>	"Democrat", "Republican", or "Other"

Column	What it holds
<code>score</code>	The NOMINATE score, from about -1 (most liberal) to 1 (most conservative)

Block 3: Summaries

Now that the data is loaded, we can summarize it to understand patterns and differences across members and parties.

Let's explore this dataset!

An overview

```
# How many rows does it have?
nrow(nominate)
```

```
[1] 31663
```

```
# How many unique members are there in the dataset?
length(unique(nominate$name))
```

```
[1] 7088
```

```
# How many states?
unique(nominate$state)
```

```
[1] "LA" "SC" "RI" "IL" "KS" "NC" "OH" "TN" "PA" "NY" "IN" "VT" "VA" "CO" "CA"
[16] "FL" "KY" "NJ" "MO" "GA" "WI" "MA" "MI" "NH" "IA" "MS" "AL" "AR" "TX" "NV"
[31] "MN" "ME" "CT" "MD" "WV" "DE" "NE" "OR" "MT" "WY" "SD" "ND" "ID" "WA" "UT"
[46] "OK" "NM" "AZ" "HI" "AK" "VI" "AS" "DC" "PR" "GU" "MP"
```

```
# Get an overview of the dataset.
```

```
glimpse(nominate)
```

Rows: 31,663

Columns: 7

```
$ year      <int> 1879, 1879, 1879, 1879, 1879, 1879, 1879, 1879, 1879, 1879, 1...
$ id        <int> 17, 51, 71, 74, 172, 229, 280, 282, 320, 345, 368, 400, 413, ...
$ name      <chr> "ACKLEN, Joseph Hayes", "AIKEN, David Wyatt", "ALDRICH, Nelso...
$ state     <chr> "LA", "SC", "RI", "IL", "KS", "NC", "OH", "TN", "PA", "NY", "...
$ district  <int> 3, 3, 1, 1, 1, 7, 14, 8, 10, 16, 13, 2, 3, 3, 23, 1, 1, 19, 3...
$ party     <chr> "Democrat", "Democrat", "Republican", "Republican", "Republic...
$ score     <dbl> -0.256, -0.308, 0.401, 0.457, 0.299, -0.604, -0.345, -0.501, ...
```

```
str(nominate)
```

'data.frame': 31663 obs. of 7 variables:

```
$ year      : int  1879 1879 1879 1879 1879 1879 1879 1879 1879 1879 ...
$ id        : int  17 51 71 74 172 229 280 282 320 345 ...
$ name      : chr  "ACKLEN, Joseph Hayes" "AIKEN, David Wyatt" "ALDRICH, Nelson Wilmarth"
"ALDRICH, William" ...
$ state     : chr  "LA" "SC" "RI" "IL" ...
$ district  : int   3 3 1 1 1 7 14 8 10 16 ...
$ party     : chr  "Democrat" "Democrat" "Republican" "Republican" ...
$ score     : num  -0.256 -0.308 0.401 0.457 0.299 -0.604 -0.345 -0.501 -0.265 0.351 ...
```

How is it in 2025?

```
# Let's get the subset first.
nominate_2025 <- subset(nominate, year == 2025)
```

```
# How many unique members?
length(unique(nominate_2025$name))
```

```
[1] 451
```

```
# What is the party distribution?
```

```
table(nominate_2025$party)
```

Democrat	Other	Republican
224	1	227

Average scores

```
# What is the general average score?  
mean(nominate_2025$score, na.rm = TRUE)
```

```
[1] 0.07386444
```

```
# What is the average score of Democrats?  
mean(nominate_2025[nominate_2025$party == "Democrat", ]$score, na.rm = TRUE)
```

```
[1] -0.3863604
```

```
# What is the average score of Republicans?  
mean(nominate_2025[nominate_2025$party == "Republican", ]$score, na.rm = TRUE)
```

```
[1] 0.5236079
```

```
# How can we get the results at the same time?  
## ifelse  
## tapply
```

```
# Use weighted.mean() to get the average score for Democrats
```

```
# Get the most liberal member of all time  
# which.max
```

Try it

[5 mins]

```

nominate <- read.csv("https://ascamara.github.io/gov50/nominate.csv")

nominate_2025 <- nominate[nominate$year == 2025, ]
# 1. Make a column called rep_indicator that is 1 for Republicans and 0 for everyone else.
#   Use it as the weights in weighted.mean() to get the Republican mean.
#   Does it match tapply()?
nominate_2025$rep_indicator <- ifelse(nominate_2025$party == "Republican", 1, 0)
mean_republican <- weighted.mean(nominate_2025$score, nominate_2025$rep_indicator, na.rm = TRUE)

cat("Mean Republican:", mean_republican, "\n")

```

Mean Republican: 0.5236079

```

# 2. The median score by party in the current House
tapply(nominate_2025$score, nominate_2025$party, median, na.rm = TRUE)

```

Democrat	Other Republican	
-0.395	0.152	0.530

```

# 3. Who is the most liberal member?
max_liberal_idx <- which.min(nominate_2025$score)
max_liberal_name <- nominate_2025[max_liberal_idx, ]$name
cat("The most liberal member (current House):", max_liberal_name, "\n")

```

The most liberal member (current House): GARCIA, Sylvia

Block 4: A first plot

This section introduces data visualization. We will use a line plot to show how party medians change over time.

Goal: Draw a graph reflecting the trend in House party medians.

1. Mapping:

1. year -> x
 2. score -> y
 3. party -> group
2. Geometric object: lines

Create an aggregated dataset

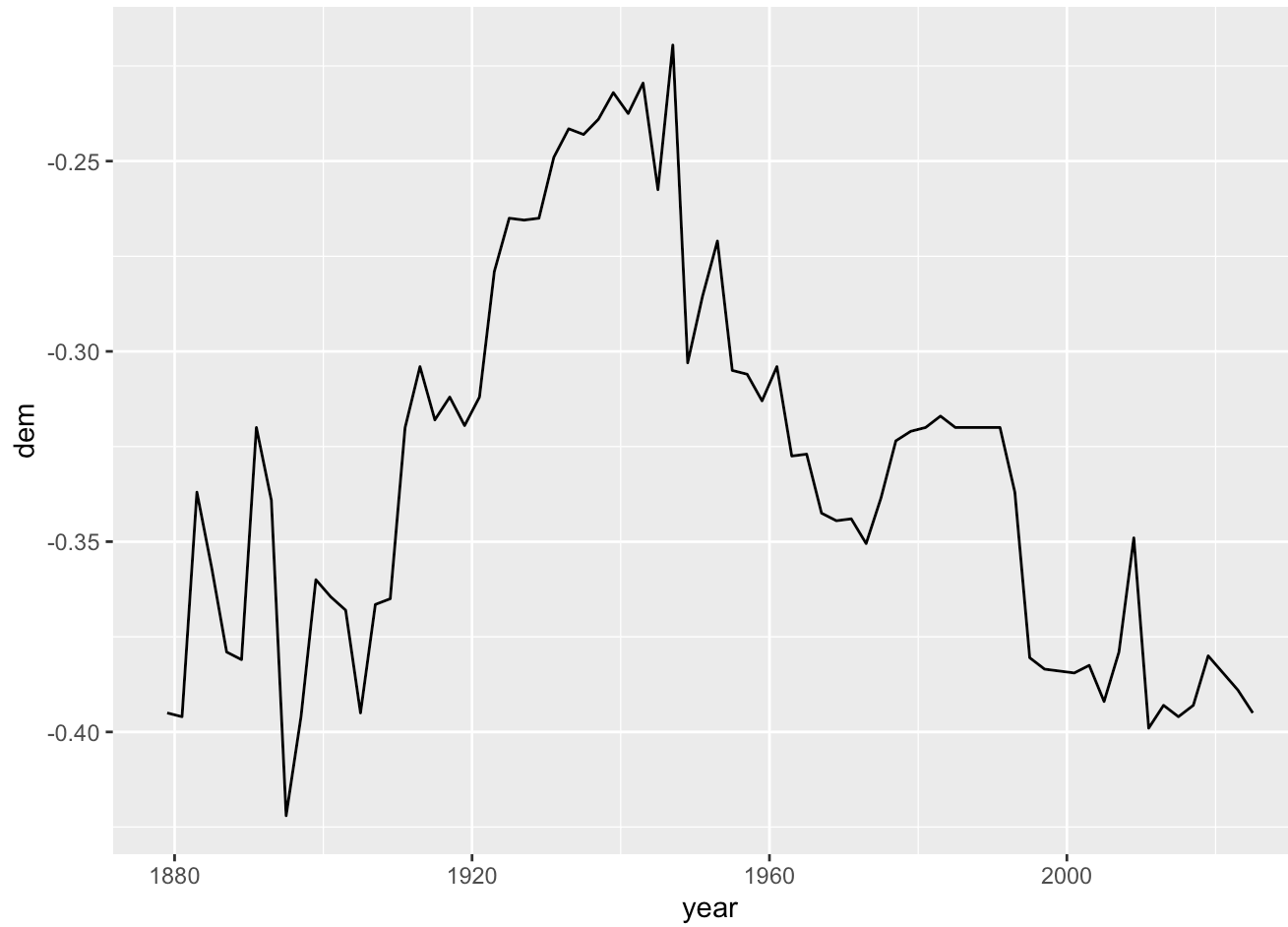
- dems
- reps
- meds

```
dems <- subset(nominate, party == "Democrat")
reps <- subset(nominate, party == "Republican")

meds <- data.frame(year = unique(dems$year),
                   dem = tapply(dems$score, dems$year, median, na.rm = TRUE),
                   rep = tapply(reps$score, reps$year, median, na.rm = TRUE)
                   )
```

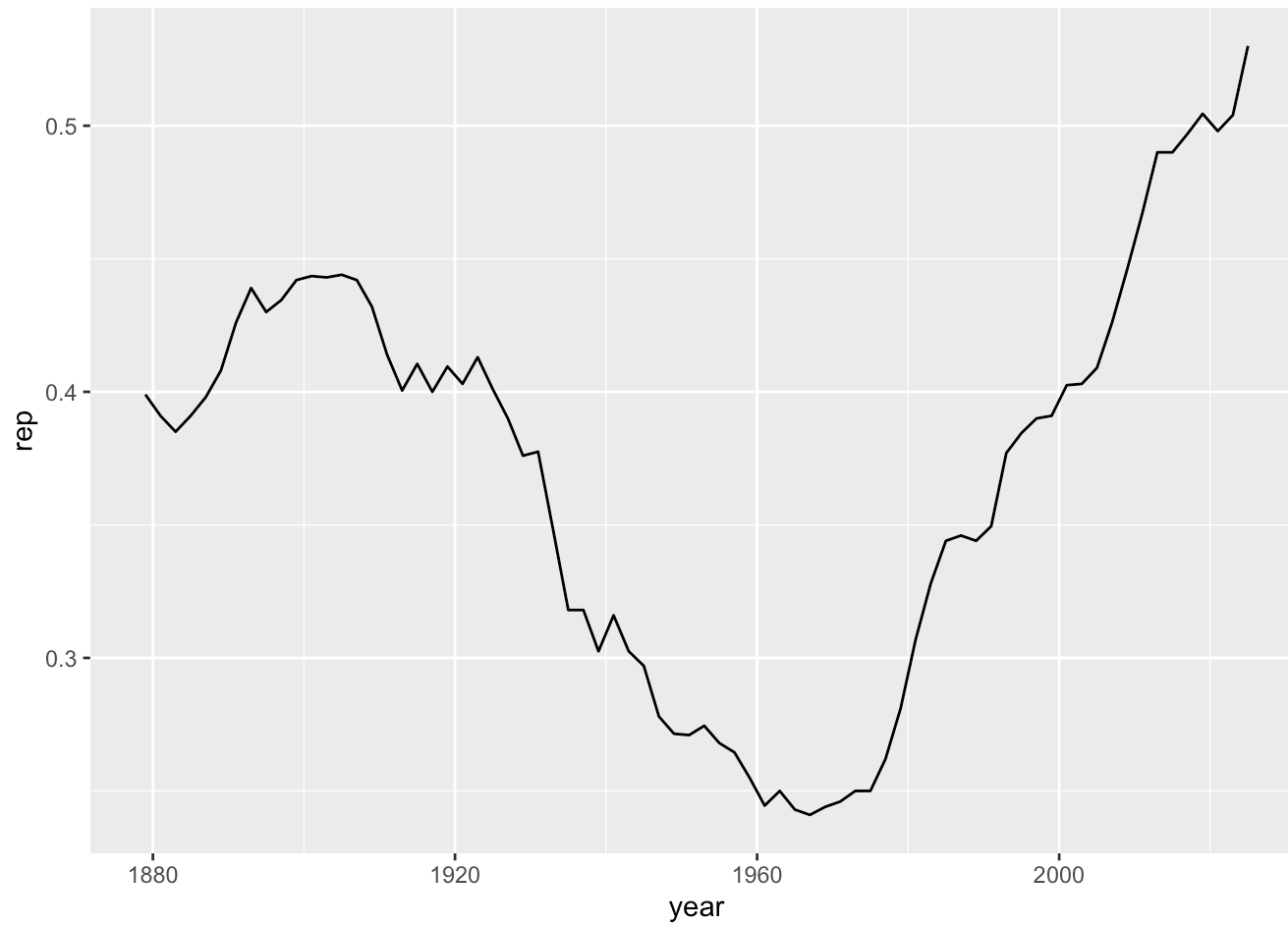
Draw a plot for Democrats

```
ggplot(data = meds, aes(x = year, y = dem)) +
  geom_line()
```



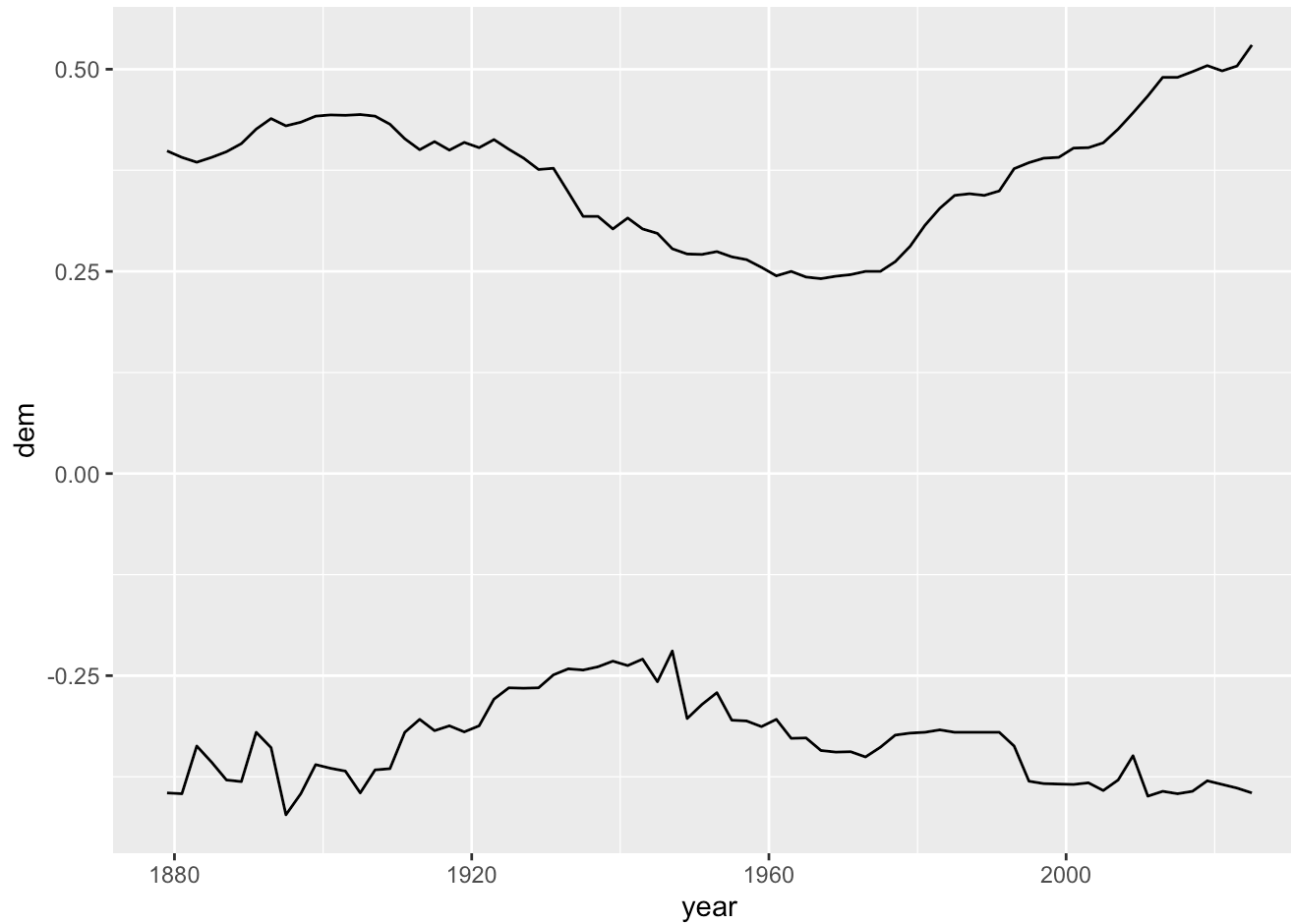
Draw a plot for Republicans

```
ggplot(data = meds, aes(x = year, y = rep)) +  
  geom_line()
```



Put them together

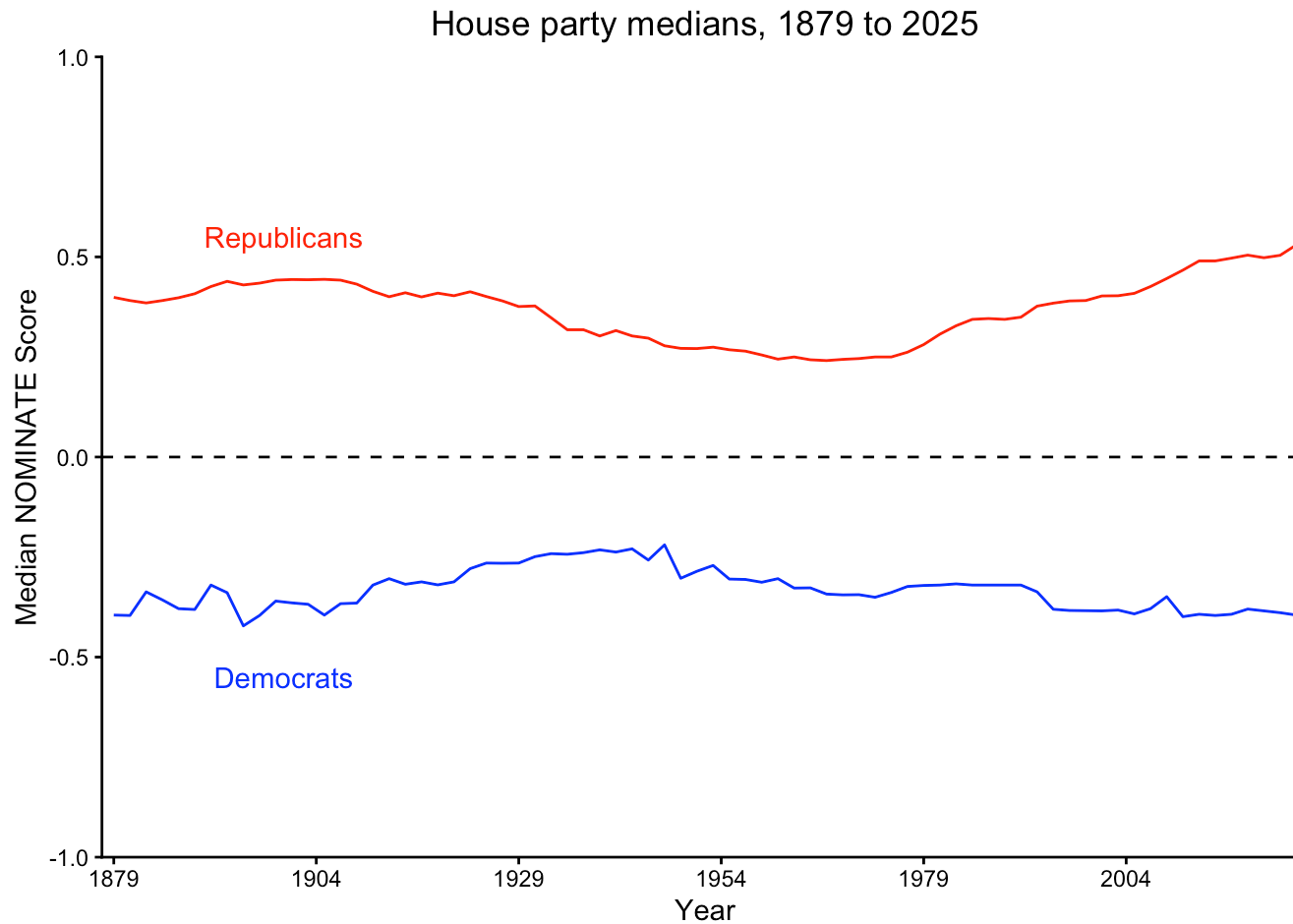
```
ggplot(data = meds, aes(x = year)) +  
  geom_line(aes(y = dem)) +  
  geom_line(aes(y = rep))
```



Prettify the graph

```
ggplot(data = meds, aes(x = year)) +  
  geom_line(aes(y = dem), color = "blue") +  
  geom_line(aes(y = rep), color = "red") +  
  geom_hline(yintercept = 0, linetype = "dashed") +  
  annotate("text", x = 1900, y = 0.55, label = "Republicans", color = "red") +  
  annotate("text", x = 1900, y = -0.55, label = "Democrats", color = "blue") +  
  scale_y_continuous(limits = c(-1, 1), expand = c(0, 0)) +  
  scale_x_continuous(breaks = seq(1879, 2025, by = 25), limits = c(1879, 2025), expand = c(0.01,  
  theme_classic() +  
  labs(x = "Year",
```

```
y = "Median NOMINATE Score",  
title = "House party medians, 1879 to 2025") +  
theme(plot.title = element_text(hjust = 0.5))
```



Try it

```
# 1. Add a column called gap to meds: the rep column minus the dem column  
meds$gap <- meds$rep - meds$dem  
  
# 2. Plot gap against year as a line with ggplot, with axis labels and a title
```

```
ggplot(data = meds, aes(x = year, y = gap)) +  
  geom_line() +  
  theme_classic() +  
  labs(x = "Year",  
       y = "Gap in NOMINATE Score",  
       title = "Gap Between Republicans and Democrats, 1879 - 2025",  
       subtitle = "Gap = Rep. - Dem.") +  
  theme(plot.title = element_text(hjust = 0.5),  
        plot.subtitle = element_text(hjust = 0.5)) +  
  scale_y_continuous(limits = c(0, 1), expand = c(0, 0))
```

